HARNESS LAB · MÓDULO 1 · AULA 1.3

Por que o agente *esquece a regra*

Estava no CLAUDE.md. Ele rodou mesmo assim. Tudo o que está no contexto disputa a mesma atenção, e uma regra escrita é um pedido. O que não pode acontecer de jeito nenhum precisa de uma cancela fora do modelo.

`Lost in the Middle` `agulha no palheiro` `context rot` `instrução demais`
`deriva de instrução` `urgência` `/context` `/autocompact` `/effort` `hook`

VOCÊ SAI COM

As suas 5 correções mais recentes ao agente anotadas em `correcoes.md`, na raiz do projeto: o que você pediu, o que ele fez e como você corrigiu. Na aula 1.4, cada linha ganha nome.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

O mesmo mecanismo que faz a LLM funcionar, a atenção, é o que cria os problemas desta aula. E é por causa dele que a gente precisa de harness, de gerenciar contexto e de escrever bons prompts.

Lost in the Middle. A atenção tem formato de parábola: tokens no começo e no fim do contexto recebem mais peso, os do meio recebem menos. Regra de vida ou morte precisa ficar no começo (o CLAUDE.md) ou ser relembrada a cada turno. O que você acabou de pedir, no fim, também pesa muito. E conforme o contexto cresce, a barriga cresce: a curva inteira achata e até o começo perde peso.



Agulha no palheiro. O teste clássico esconde um trecho num texto longo e pede para o modelo achar. Os modelos antigos sofriam, os atuais acham bem. Mas ainda sofrem para ligar pontos espalhados em vários lugares. No benchmark MRCR v2 mostrado na aula, o acerto do Opus 5 cai conforme o contexto cresce, de 8 mil para 128 mil e para 512 mil tokens. A recomendação da aula: trabalhar até uns 300 mil tokens, com os gates e a estrutura de harness do curso.

Degradação da inteligência. Sessão nova é o auge. Conforme o contexto cresce e suja, a qualidade cai, porque tudo o que está ali compete por atenção: cada token é comparado com todos os outros. Se a sua sessão chegou na compactação automática do modo padrão, faz tempo que regra estava sendo ignorada. **Instrução demais** piora: harness inflado é regra demais disputando o mesmo espaço.

Estava escrito. Ele rodou. "Nunca rode `migrate:fresh`" no CLAUDE.md, e em algum momento ele roda (em Node, `prisma migrate reset`; em Rails, `rails db:reset`). Dois motivos: a conversa cresceu tanto que a primeira linha perdeu peso para o que veio no fim, e a **deriva de importância**. É a placa de proibido virar à esquerda: o caminho da direita está bloqueado, ele tenta contornar, contornar, até que a regra dilui e ele vira. Na demo, com contexto limpo, o Claude recitou a regra inteira sem abrir o arquivo. Em sessão avançada, a mesma regra pode ser abandonada.

Urgência. Banco travado, banco inconsistente, informações erradas: a urgência puxa contra a regra até derrubar. **Pensar mais não resolve:** aumentar o esforço faz o modelo pensar mais em cima de contexto sujo. Esforço é para profundidade de análise e de plano, não para qualidade. A saída é a cancela: regra determinística, em código, que barra a ação e explica o porquê. É o Módulo 2, de hooks.

Posição pesa

Começo e fim levam mais atenção. O que é crítico mora no CLAUDE.md, curto, ou volta a cada turno.

Tamanho pesa

Quanto maior o contexto, mais tudo dilui, inclusive o começo. Sessão nova é o auge.

Saber não é cumprir

Ele pode recitar a regra e quebrar mesmo assim, sob pressão. Aí não é falta de informação.

Pedido depende de atenção

E a atenção é disputada. O que não pode acontecer vira trava fora do modelo.

> *"Claude treats them as context, not enforced configuration. To block an action regardless of what Claude decides, use a PreToolUse hook instead."*
>
> Documentação oficial do Claude Code · Memory (sobre CLAUDE.md e auto memory)

2 O que a aula *não teve tempo de dizer*

ALÉM DA AULA · A VERSÃO FÁCIL FOI CORRIGIDA. A DIFÍCIL, NÃO.

A agulha no palheiro clássica é casar palavra: a pergunta usa as mesmas palavras do trecho escondido. O NoLiMa (ICML 2025) tirou as palavras em comum: o modelo precisa inferir. Doze modelos que anunciam janela de 128 mil tokens ou mais. Com 32 mil, **10 de 12** caíram para menos da metade do próprio desempenho em texto curto. O GPT-4o foi de 99,3% para 69,7%. A Chroma (Context Rot, 2025) mediu 18 modelos, Claude 4 entre eles: o desempenho cai com o tamanho da entrada até em tarefa trivial, e os **distratores**, trechos parecidos com a resposta, pioram o resultado. No seu repositório: dois services quase iguais, a migration velha e a nova, o teste que ninguém apagou.

Fontes: Modarressi et al., arXiv 2502.05167. Hong, Troynikov e Huber, research.trychroma.com/context-rot. A curva em U original: Liu et al., 2023, arXiv 2307.03172, com modelos da época (GPT-3.5, Claude 1.3).

ALÉM DA AULA · MAIÚSCULA E "IMPORTANTE" NÃO COMPRAM ATENÇÃO

Um estudo de 2026 mudou tamanho, posição e estrutura do CLAUDE.md e do AGENTS.md: a obediência não mudou de forma detectável. O que variou foi o momento da sessão. Já o volume pesa: no IFScale (NeurIPS 2025), com 500 instruções ao mesmo tempo, o melhor modelo acertou 68%, e as instruções do começo da lista foram mais seguidas. A documentação vai no mesmo sentido: "target under 200 lines per CLAUDE.md file. Longer files consume more context and reduce adherence." Cada regra que você cola depois de uma correção divide a atenção com as outras. Por enquanto, só anote. Enxugar o arquivo é o Módulo 3.

Fontes: McMillan, arXiv 2605.10039. Distyl AI, arXiv 2507.11538. Memory, "Write effective instructions".

ALÉM DA AULA · A DEMO PROVA QUE ELE SABE. NÃO PROVA QUE ELE CUMPRE.

Perguntar a regra num turno só, com contexto limpo, quase sempre dá certo. O Asymmetric Goal Drift (2026) pôs agentes de código no OpenCode com uma regra no system prompt e um ambiente que pressionava contra ela (GPT-5 mini, Claude Haiku 4.5, Grok Code Fast 1). A violação cresceu conforme o contexto acumulava e a pressão aumentava. A conclusão dos autores: checar obediência num turno só não basta. Outro estudo viu que seguir bem a hierarquia de instruções não prevê resistência à deriva. Por isso a lição de casa olha as suas sessões reais, não um teste rápido.

Fontes: Saebo et al., arXiv 2603.03456. Menon et al., arXiv 2603.03258.

ALÉM DA AULA · RELEMBRAR A REGRA: O QUE O CLAUDE CODE JÁ FAZ POR VOCÊ

Depois de um `/compact`, manual ou automático, o CLAUDE.md da raiz do projeto é relido do disco e volta inteiro para o contexto. O que você combinou só na conversa vira resumo, e as rules com `paths:` só voltam quando ele mexer num arquivo que casa. Regra combinada no chat que precisa durar vai para o CLAUDE.md. Pedir para ele repetir a regra antes de agir vale como analogia, não como resultado medido: num estudo de 2025, recitar a evidência da tarefa antes de responder ("recite then solve") reduziu a queda causada pelo contexto longo. O estudo não mediu obediência a regra. Para relembrar a cada turno sem depender de você, existe hook: o texto que um `UserPromptSubmit` imprime entra no contexto. É Módulo 2.

Fontes: Context window, "What survives compaction". Du, Tian, Peng et al., arXiv 2510.05381. Hooks reference, "Exit code output".

ALÉM DA AULA · A COMPACTAÇÃO AUTOMÁTICA CHEGA TARDE: MUDE O PONTO

No Opus 5.5, com janela nativa de 1 milhão de tokens, a compactação automática do padrão roda por volta de 967 mil. É o "você já estava ferrado" da aula. O `/autocompact` muda esse ponto para o modelo atual e salva para as próximas sessões: `/autocompact 300k` põe a compactação na janela de trabalho que a aula recomenda (aceita de 100k a 1M; `/autocompact auto` volta ao padrão). Antes de uma tarefa nova numa sessão longa, `/compact` com um foco escolhe o que o resumo guarda. E o `/context` mostra o tamanho atual e quais arquivos de memória carregaram.

Fonte: Model configuration, "Default auto-compact thresholds" e "Set the auto-compact window".

ALÉM DA AULA · OS NOMES CERTOS DO ESFORÇO, E POR QUE ELE NÃO É TRAVA

O "baixo, médio, alto" da aula é o `/effort`: `low`, `medium`, `high`, `xhigh` e `max`. No Opus 5.5, o padrão é `medium`. A palavra `ultrathink` no pedido aumenta o raciocínio só daquele turno. O Ultracode, no mesmo menu, não é nível de esforço: liga a orquestração de workflows. Nenhum deles tira o que já está no contexto. Pesquisadores do programa Anthropic Fellows mostraram que raciocínio mais longo pode reduzir o acerto, e que os modelos Claude ficam mais distraídos por informação irrelevante quando pensam mais.

Fontes: Model configuration, "Adjust effort level". Gema, Hägele et al., arXiv 2507.14417.

3 Faça no *seu repositório*

A lição de casa da aula: gerar um resumo das suas sessões, achar as últimas correções que você fez ao agente e anotar 5. Esta aula não instala hook nem regra. Instala o arquivo da lista e um plugin que lê o seu histórico e preenche a lista com você. Precisa de `jq` (`jq --version` responde? Se não: `sudo apt install jq` ou `brew install jq`).

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou numa aula anterior? Só atualize:

Terminal · qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Instale o arquivo da lista no seu projeto.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 1.3
```

Cria `correcoes.md` na raiz, com a tabela vazia. Se você já tinha um arquivo com esse nome, o seu fica intacto e o da aula vai para `correcoes.md.harness-lab-novo`.

3 Instale o plugin `correcoes`.

Terminal · qualquer pasta — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin marketplace update harness-lab
claude plugin install correcoes@harness-lab
```

Prefere fazer de dentro do Claude Code? O mesmo, em três comandos, um por mensagem, na ordem. Quem já adicionou o marketplace numa aula anterior pula a primeira linha.

Dentro do Claude Code — sessão

```
/plugin marketplace add beerandcodeteam/harness-lab-materiais
/plugin marketplace update harness-lab
/plugin install correcoes@harness-lab
/reload-plugins
```

4 Rode a skill na raiz do projeto.

Abra o Claude Code na pasta onde você costuma trabalhar com ele e chame a skill. Ela roda o script do plugin, que lê o que você digitou nas últimas 5 sessões deste projeto, mostra as frases com cara

de correção, escolhe as 5 mais recentes e preenche o `correcoes.md`. Ela não classifica nada e marca com `?` o que não deu para deduzir.

Dentro do Claude Code · na raiz do projeto — sessão

```
/correcoes:anotar
```

Achou poucas? Rode de novo olhando mais sessões (`10`) ou todas as mensagens (`--todas`). É um ou outro, numa mensagem nova:

Dentro do Claude Code · um ou outro — sessão

```
/correcoes:anotar 10
/correcoes:anotar --todas
```

5

Confira as mensagens você mesmo, no terminal.

O mesmo script roda fora do Claude Code, direto do repositório baixado. É só leitura. Revise o que a skill escolheu e ache o que o filtro por palavra deixou passar ("na verdade era X", "use Y em vez de Z").

Terminal · na raiz do seu projeto — shell

```
S=~/harness-lab-materiais/plugins/correcoes/skills/anotar/scripts
"$S"/mensagens.sh
"$S"/mensagens.sh 10 --todas
```

exemplo de saída — * = cara de correção · a linha sem * é o pedido anterior

```
# 5 sessão(ões) deste projeto · 41 mensagens suas · 6 com cara de correção (*)

## sessão 3caf65df · 2026-10-30 14:02 a 15:40 · 9 mensagens
  14:02  adicione o campo status no pedido, com migration
* 14:31  para tudo! você rodou migrate:fresh e apagou o banco
  15:10  refatore o CarrinhoService
* 15:22  volta, você mudou a assinatura pública
```

Quer o contexto inteiro de uma sessão, com o que o agente respondeu? Abra pelo seletor de sessões. Ele abre uma sessão interativa:

Terminal · na raiz do seu projeto — claude

```
claude --resume
```

6

Use o que você já tem das aulas anteriores, se tiver.

Rodou o diagnóstico da aula 1.1? As mensagens que ele marcou como "testou" e "repetiu" são candidatas fortes. E o `/insights` gera um relatório HTML das suas sessões recentes nesta máquina, com os pontos de atrito: pedido mal entendido, código com bug. Ele gasta tokens do seu plano. Um comando por mensagem:

Dentro do Claude Code — sessão

```
/diagnostico-operador listar testou repetiu
/insights
```

7 **Confira o arquivo.**

Terminal · na raiz do seu projeto — shell

```
grep -nE "^\| [1-5] \|.*\| \| " correcoes.md
```

O comando lista as linhas de 1 a 5 que ainda têm alguma coluna vazia (fora a "Camada", que fica vazia mesmo). Logo depois do passo 2, ele imprime as 5. **Terminou quando não imprimir nada.** Abra o arquivo e troque cada `?` que a skill deixou pelo que você lembra: quando, o que pediu, o que ele fez, como corrigiu. Depois, poste as suas correções no comentário da aula, como o Lucas pediu: sem nome de cliente, sem segredo.

8 **Opcional: o teste da aula no seu projeto.**

Tem uma regra dura no CLAUDE.md? Confira que ela está lá e, numa sessão nova, pergunte. Troque `migrate:fresh` pela palavra da sua regra (`migrate reset` no Prisma, `db:reset` no Rails). Se o grep não imprimir nada, a regra não está nesse arquivo. O `/context` mostra o CLAUDE.md entre os arquivos de memória carregados. Um comando por mensagem:

Terminal · na raiz do seu projeto — shell

```
grep -n "migrate:fresh" CLAUDE.md
```

Dentro do Claude Code · sessão nova — sessão

```
/context
> Qual a regra sobre usar migrate:fresh neste projeto?
```

Ele recitar prova que a regra está no contexto, não que ele vai cumprir na hora da pressão (seção 2).

9 **Opcional: compacte antes de chegar na barriga.**

Vale para este modelo nas próximas sessões. Para voltar ao padrão: `/autocompact auto`.

Dentro do Claude Code — sessão

```
/autocompact 300k
```

O arquivo que o instalador colocou

correcoes.md · trecho — instalado pelo instalar.sh · preenchido pela skill ou por você

```
# Correções ao agente

| # | Quando | O que você pediu | O que o agente fez | Como você corrigiu | Camada (aula 1.4) |
| --- | --- | --- | --- | --- | --- |
| ex. | 2026-11-02 | Adicionar o campo `status` no pedido, com migration
      | A migration falhou, ele disse que o banco estava inconsistente e rodou
        `migrate:fresh` | Mandei parar, recriei os dados de teste e pedi para
        ele corrigir só a migration | |
| 1 | | | | | |
... até a 5, e as perguntas para cada linha
```

A coluna "Camada" é da aula 1.4: lá você classifica cada linha em Regra, Trava, Prova ou Loop, ou conclui que faltou inteligência mesmo, e o plugin da 1.4 lê este mesmo arquivo. No fim dele, quatro perguntas para anotar sem responder ainda: a regra estava escrita? Ele saberia recitar? A sessão era longa? Ele disse que tinha terminado?

A skill `anotar` é manual (`disable-model-invocation: true`): o Claude não dispara sozinho. O script é chamado pelo `${CLAUDE_SKILL_DIR}`, que aponta para a pasta da skill onde quer que o plugin esteja instalado, e o `allowed-tools` libera esse script, o `git log` e a leitura de arquivos sem pedir permissão.

4 Arquivos que *reforçam a aula*

`correcoes.md`

A lição de casa em forma de arquivo: 5 linhas, três colunas, a coluna da aula 1.4 vazia. Fica no repositório, então a 1.4 parte dele e não da sua memória.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.3-por-que-o-agente-esquece-a-regra/projeto/

`plugins/correcoes/skills/anotar/SKILL.md`

"Gera um resumo das suas sessões e acha as últimas correções", virado skill: lê o histórico, escolhe as 5 mais recentes, não inventa e não classifica.

No repositório: plugins/correcoes/skills/anotar/

`scripts/mensagens.sh`

Lê o `~/.claude/history.jsonl` e separa por projeto e por sessão. Mostra que as correções estão no seu histórico, não na sua lembrança. Testado com histórico falso no `teste/verificar.sh`.

No repositório: plugins/correcoes/skills/anotar/scripts/

`modelos/claude-md-nunca-faca.md`

A regra dura da demo, escrita do jeito que a documentação pede (concreta, curta, no topo), com os equivalentes em Prisma, Rails e Django. E o aviso: é pedido. A cancela é a aula 2.3.

No repositório: modulo-1-o-operador-e-o-trilho/aula-1.3-por-que-o-agente-esquece-a-regra/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`/correcoes:anotar` não aparece no menu	Plugin instalado com a sessão já aberta, ou marketplace não adicionado	`/reload-plugins`. Se ainda não aparecer, refaça o passo 3 e confira com `claude plugin list`
"Nenhuma mensagem deste projeto"	O histórico é separado pela pasta onde você abriu o Claude Code. Você rodou em outra pasta	Rode na raiz do projeto onde trabalhou. Sessões abertas em subpastas dele entram junto
Sessões antigas sumiram do `claude --resume`	O Claude Code apaga os transcripts com mais de 30 dias (`cleanupPeriodDays`)	Faça a lição esta semana. O `history.jsonl`, que o script lê, guarda só o que você digitou
Chamar toda regra quebrada de lost in the middle	É o nome mais famoso, mas o CLAUDE.md entra no começo do contexto	Pergunte a regra. Se ele recita, a informação estava lá: anote "sabia e fez mesmo assim"
Reescrever a regra em maiúscula e colar mais regra a cada correção	Estrutura não mudou a obediência no estudo que mediu; volume dilui	Só anote. O que não pode acontecer vira hook (aula 2.3); enxugar é o Módulo 3
Classificar as correções já nesta prática	Vontade de resolver logo	Descreva o que aconteceu. O nome de cada uma vem na aula 1.4

6 Você sai *com*

- [ ] `correcoes.md` na raiz do projeto com as 5 correções mais recentes: o que pediu, o que ele fez, como corrigiu.
- [ ] O plugin `correcoes` instalado e o `/correcoes:anotar` funcionando no seu projeto.
- [ ] A diferença entre ele não ter a informação e ele saber e quebrar mesmo assim.
- [ ] Opcional: o ponto da compactação automática onde você quer, conferido com o `/context`.
- [ ] As suas correções postadas no comentário da aula.

PROMETIDO NA AULA

- ✓ "Gera um resumo das suas sessões, acha as últimas correções e anota": o plugin `correcoes`, com a skill `/correcoes:anotar` e o script `mensagens.sh` (seção 3, passos 3 a 5). O `/insights` como relatório das sessões (passo 6).
- ✓ "Cinco correções anotadas": a tabela `correcoes.md`, instalada com `instalar.sh 1.3` (passo 2) e lida pela aula 1.4.
- ✓ "Coloca no comentário da aula": passo 7.
- ✓ "Você vai ver alguns artigos" sobre a agulha no palheiro: NoLiMa, Context Rot e Lost in the Middle (seções 2 e 8).
- ✓ Regra "relembrada a cada turno" e "pedir para lembrar alguma regra": o que o `/compact` recarrega, o "recite then solve" e o hook que relembra (seção 2).
- ✓ "Travar isso em linha de código" no próximo módulo: a regra da demo em `modelos/claude-md-nunca-faca.md`, com o aviso de que a cancela é a aula 2.3.

7 *Glossário*

Lost in the Middle	O modelo usa melhor o começo e o fim de um texto longo do que o meio (curva em U).
Agulha no palheiro	Needle in a haystack: esconder um trecho num texto longo e pedir para achar. A versão literal os modelos atuais resolvem.
Context rot	Queda de qualidade conforme o contexto cresce, mesmo em tarefa simples.
Distrator	Trecho parecido com a resposta certa, mas errado. Piora o resultado em contexto longo.
Deriva de instrução	Goal drift: o agente conhece a regra e a viola conforme o contexto acumula e a pressão cresce.
Compactação	O Claude Code troca a conversa por um resumo para caber na janela. O CLAUDE.md da raiz é relido do disco.
Esforço	O quanto o modelo raciocina por passo (`/effort`). Profundidade, não obediência.
Hook	Programa que o Claude Code roda fora do modelo em pontos da sessão. Pode barrar uma ação. Módulo 2.

8 Para *ir além*

- Lost in the Middle (Liu et al., 2023): a curva em U original e o experimento de mudar a resposta de posição.
- NoLiMa (ICML 2025): a agulha sem palavra em comum, e a tabela de queda por modelo e por tamanho.
- Asymmetric Goal Drift in Coding Agents (2026): o cenário de pressão contra o system prompt e por que um turno só engana.
- Memory: "Write effective instructions", tamanho do CLAUDE.md e "Instructions seem lost after /compact".
- Context window: o que carrega antes do primeiro pedido e a tabela "What survives compaction".

Próxima: *Aula 1.4 · As 4 camadas do harness*

As suas 5 correções ganham nome: faltou informação (Regra), faltou obrigar (Trava), faltou checar (Prova), faltou continuar sozinho (Loop) ou faltou inteligência mesmo. E a regra do `migrate:fresh` ganha a cancela dela na aula 2.3.